

面试押题：量化 Runtime (AWQ W4A16) 与 BI

Router GEMM 三级后端

面向：算子 / 推理引擎 / 量化方向的技术面。两块都是简历上"能深挖"的硬骨头。

口径原则：只讲我真做过、能对上代码和实测数据的；边界和没做的部分主动说清，不吹。

关联材料：[面试押题_Mooncake] 同一批秋招准备；R3 / FlashInfer / OE 复盘另有单独文档。

§0 两句话定位（开场自我锚定）

- **量化 Runtime (LLMQRT)**：一个 W4A16 AWQ 为主的 **单卡** 量化推理 runtime (PyTorch C++/CUDA Extension)。我做的是 **AWQ 的 GEMV/GEMM CUDA kernel**——按 batch 形状分派 GEMV(decode) / GEMM(prefill)，写了 coalesced 访存、double-buffer、split-K 几个变体，并用 Compute Sanitizer 定位修了一个 **IC 不整除时的越界读**。端到端 Qwen3-8B prefill 吞吐 $6.06\times$ 。
- **BI Router GEMM 三级后端**：在 vLLM 的 **Batch-Invariant (逐 bit 确定性)** 模式下，MoE 的 router gate GEMM $[M, 2048] \times [2048, 128]$ 在 decode 小 M 下用 persistent kernel 只跑 1 个 CTA、SM 占用 0.4%。我做了一条 **DeepGEMM $\rightarrow$ Triton full-K $\rightarrow$ persistent** 的按 dtype/M 静态分派链，在保持逐 bit 确定性的前提下把 router GEMM 时间打下来。**关键诚实点：它是 opt-in (默认仍 persistent)**，因为 DeepGEMM 相对优化后 full-K 只有 $1.127\times$ ，没过我自己设的 $1.2\times$ 默认晋级门槛。

红线（必须记牢，别在面试里说漏）

- LLMQRT 是**单卡** runtime，**不做 TP**（代码里 tensor_parallel/all_reduce 全在 3rdparty/cutlass，不是本项目的）。如果面到 AWQ 的 TP 切分（Q/K/V 列切、O/down 行切 + all-reduce），那是在 **别的项目（vLLM 侧）** 的经验，要分清是哪个项目，别混。
- LLMQRT 我拿到的是 **depth-1 浅克隆、单 squash commit**，所以我**不能说**"通过 git 定位到修复 commit"。越界的根因和 fix 我能从代码现状 + Compute Sanitizer 讲清，但"修复前长什

么样"的 git 时间线在这份克隆里看不到，需要 unshallow 才有。面试就说"根因清楚、现状带保护"，不假装有 git 证据链。

第一部分：量化 Runtime (AWQ W4A16)

§1.1 背景速成：AWQ / W4A16 是什么

- **W4A16**: 权重 4-bit (int4) 量化、激活保持 fp16。显存降到约 1/4，decode 是 memory-bound，权重搬得少 → 快。
- **AWQ (Activation-aware Weight Quantization)**: 不是均匀量化所有权重，而是按激活幅度挑出"重要通道"用 per-group scale 保护，量化误差集中到不重要通道。落到 runtime 层面，我关心的是它的 **存储布局** 和 **反量化 + GEMM kernel**，不是量化算法本身（那是离线校准阶段的事）。
- **group-wise 量化**: `group_size=128`，即每 128 个 input channel 共享一组 scale/zero。存储：
- `qweight: [in_features, out_features // 8]`，int32，每个 int32 打包 8 个 int4。
- `qzeros: [in_features // group_size, out_features // 8]`，int32。
- `scales: [in_features // group_size, out_features]`，fp16。

§1.2 我做的工程全景

模块	文件	我做/理解的点
GEMV 基础	<code>csrc/awq/gemv.cu</code> <code>gemv_kernel_g128</code>	decode 路径（小 batch）。每 warp 负责 1 个 OC，IC 分 1024 一块的 tile 累加。修了 IC 不整除的越界。
GEMV coalesced	<code>csrc/awq/gemv_coalesced.cu</code>	合并访存版 + v2 缓存 scale/zero 减少重复 load (<code>cached_scales[8]/cached_zeros[8]</code>)
GEMM (mma)	<code>csrc/awq/gemm.cu</code>	prefill 路径。Tensor Core <code>mma.sync</code> +

模块	文件	我做/理解的点
	<code>gemm_forward_4bit_cuda_m16n128k32</code>	<code>ldmatrix</code> , shared mem 打 padding 避 bank conflict, split-K
GEMM double-buffer	<code>csrc/awq/gemm_db.cu / gemm_db_m32.cu</code>	shared mem + register 双缓冲, overlap load 与 compute (ping-pong)
快速反量化	<code>csrc/awq/dequantize.cuh</code>	<code>dequantize_s4_to_fp16x2</code> : lop3 位技巧, int4→fp16 单指令
dispatch	<code>nn_models/.../linear_awq.py</code> <code>WQLinearMMFunction</code>	<code>numel/hidden<8 && seq==1</code> → GEMV, 否则 GEMM
数值验证	<code>unittest/test_w4a16_awq_gemm.py</code>	PyTorch 反量化 reference 对拍, fp16 <code>ATOL=1e-3</code>

实测 (`LLMQRT-perf-test-log.txt` / profile 日志, Qwen3-8B, H200 pytorch 25.04 + cu12.9)

- 端到端 prefill 吞吐 $355.09 \rightarrow 2150.70 \text{ tok/s} = 6.06\times$; prefill 延迟 $183.05\text{ms} \rightarrow 30.22\text{ms} = 6.05\times$ 。
- double-buffer 对 GEMM 的增量: prefill $79.7\mu\text{s} \rightarrow 73.9\mu\text{s} = 1.08\times$ 。
- profile 里 `gemv_kernel_g128` 是第 2 耗时 kernel (310ms / 1620 calls, avg 0.19ms), `WQLinearMMFunction` 第 1 (476ms / 1800 calls)。
- **decode 吞吐几乎不变** ($23.88 \rightarrow 23.76 \text{ tok/s}$) ——这条要主动说: AWQ 的收益在 prefill (compute/带宽都吃满), decode 单 token memory-bound、batch=1 时 GEMV 本就轻, KV cache 搬运才是 decode 瓶颈, 不是 router/gemv。别把 prefill 的 $6\times$ 说成端到端 $6\times$ 。

§1.3 深挖题①: `gemv_kernel_g128` 的越界读——根因、定位、修复

一句话: GEMV kernel 用向上取整算迭代次数, 当 `IC` 不是每轮处理量 (1024) 的整数倍、或

`IC/group_size` 不是每轮组数 (8) 的整数倍时, 尾轮 iteration 有部分线程会读到 `inputs/weight` buffer 之外, Compute Sanitizer 报 invalid global read。修复是在两处 load 前加 `< IC` 的 predicate。

根因链（能对着代码逐行讲）

kernel 的 tile 划分 (`gemv.cu:82-84`) :

```
const int IC_per_warp_iter = WARP_SIZE * 4 * PACK_FACTOR;    // 32 线程×4float4×8half = 1024
IC/轮
const int groups_per_iter  = IC_per_warp_iter / group_size; // 1024/128 = 8 组/轮
const int num_iterations   = make_divisible(IC/group_size, groups_per_iter); // 向上取整!
```

`make_divisible(c,d) = (c+d-1)/d (:28)` 是 **ceiling 除法**。所以只要 `IC/group_size` 不能被 8 整除（等价地 `IC` 不是 1024 的整数倍），`num_iterations` 会多算一轮。多出来的那轮里, 线程按 `inputs_ptr_delta = iter_idx*WARP_SIZE*4 + threadIdx.x*4 (:112)` 算出的偏移已经越过了真实数据边界。

- 越界读 A: `*(float4*)packed_inputs = *(inputs_ptr + ic_0) (:121)` ——读 `inputs` 尾部之外。
- 越界读 W: `*(weight + ic_actual*weight_stride + packed_oc_idx) (:131)`, `ic_actual=(inputs_ptr_delta+ic_0)*8+ic_1 (:127)` ——读 `weight` 尾部之外。

为什么"读越界"还常常能出正确数字、难发现: 越界地址往往落在同一块大 allocation 的相邻 tensor 或 pad 区里, 值是"脏但有限"的, psum 里加进去的是一小段噪声; 只有当 `IC` 恰好卡在边界、且脏值恰好非零时结果才偏。所以功能测试 (`IC=256、512` 这种 1024 因子) 根本触发不到——这类 bug 靠 Compute Sanitizer 的 `memcheck` 才抓得到, 不是靠对拍 `max_err`。

定位手段: Compute Sanitizer

```
compute-sanitizer --tool memcheck python awq_run_qwen2.py
```

`memcheck` 会精确报出 "Invalid __global__ read of size 16 bytes" + 出错的 kernel 名 + PC + 线程/block 坐标。看坐标就知道是尾轮的高 `threadIdx.x` 线程越界, 回到 `num_iterations` 的 ceiling 就定位到根因。

修复

两道 in-kernel predicate (`gemv.cu:120` 外层 float4 load、`:129` 内层 per-IC) :

```

if (inputs_ptr_delta + ic_0 < IC / PACK_FACTOR) {    // :120 保护 float4 读
    ...
    if (ic_actual < IC) {                            // :129 保护 weight 读 + 累加
        ...
    }
}
}

```

`gemv_coalesced.cu` 是同一 bug 类，同样加了 guard，而且多一道 `group_idx < IC/group_size (:183)`，把 scale/zero 的 group 索引也 predicate 掉。`runtime_refact` 树的 gemv 与 tutorial 逐字节一致（只差注释）。

诚实边界（面试主动说）：我这份仓是浅克隆、单 commit，git 里看不到"修复前"版本；我讲的根因是从代码现状 + Compute Sanitizer 报错形态反推的，逻辑闭环但没有 git diff 佐证。如果面试官要 diff，我会说"要 unshallow 或看 GitHub 历史"。

追问预案

- **Q：为什么不直接要求 `IC % 1024 == 0` 让上层 pad？** A：pad 会改 `in_features`，牵动 scale/zero/qweight 三个 tensor 的形状和整个 checkpoint 布局，代价大；in-kernel predicate 只在尾轮多几条判断，overhead 可忽略（尾轮线程本就要 early-out）。
- **Q：predicate 会不会引入 warp divergence 掉性能？** A：只有最后一轮的一部分线程走 false 分支，前面所有满轮都是全 true，divergence 只在末尾一轮、且是 reduce 前，实测 kernel 时间没有可见回退。
- **Q：越界读会不会读到别的进程内存/段错误？** A：CUDA 里越界读通常落在同一 context 的显存里（除非跨 allocation 边界到未映射页才 fault），所以更常见是"静默错值"而非崩溃——这正是它危险的地方。

§1.4 深挖题②：int4 → fp16 的 lop3 快速反量化 + order_map 交错

一句话：反量化不是"取 4bit → 转 int → 转 float"这条慢路，而是用 `lop3.b32` 位运算把 int4 直接塞进 fp16 的尾数位，配合一个 `0x64006400` magic number，一条指令解 2 个数；权重打包时用 `[0,2,4,6,1,3,5,7]` 的交错顺序，正是为了配合这个 unpack。

机制 (`dequantize.cuh dequantize_s4_to_fp16x2`)

- fp16 里 `1024.0 = 0x6400`。把一个 4-bit 值 `v` 放进 `0x6400 | v` 的低位, 这个 fp16 数就等于 `1024 + v`, 再 `sub 1024` 就还原出 `v` (浮点)。
- `lop3.b32` (3 输入查找表逻辑, `immLut = (0xf0&0xcc)|0xaa`) 一条指令完成 `(i4s & MASK) | 0x64006400`, 同时处理 packed 在一个 32bit 里的 2 个 int4 (`0x000f000f` 掩两个 lane)。
- 高低半用 `top_i4s = i4s >> 8` 复用一次移位, 4 次 `lop3` 解出 8 个 int4。
- 最后 `sub.fp16x2` (elt 01/45) 和 `fma.rn.fp16x2 × 1/16 - 64` (elt 23/67, 因为它们在高 4 bit, 等价乘 1/16 再平移) 把 8 个值一次性归位。这就是为什么 pack 顺序是交错的 `[0,2,4,6,1,3,5,7]`——让"低半字节组"和"移位 8 位后的组"各自连续, 减少 shuffle。
- runtime 侧 (`linear_awq.py from_linear`) pack 用 `order_map=[0,2,4,6,1,3,5,7]`, kernel 侧 (`gemv.cu get_reverse_order_map`) 用 `reverse=[0,4,1,5,2,6,3,7]` 取回原始 OC 位置——一个是"存进去的排列", 一个是"读出来的逆排列", 两者必须成对。

追问预案

- Q: 为什么用无符号 int4 [0,15] 而不是有符号 [-8,7]? A: 塞进尾数需要非负; 负数由 dequant 公式 `scale*(w - zero)` 里的 zero point 承担, kernel 里 `current_zeros` 就是这个。注释里 FT 作者也提到"用无符号并在减法里补偿"。
- Q: 这套 magic number 对 bf16 也能用吗? A: 不能直接照搬, bf16 指数/尾数位宽不同, magic 常数要换; 这个 kernel 是 fp16 专用 (`x` 进来强转 fp16)。

第二部分: BI Router GEMM 三级后端

§2.1 背景速成: Batch-Invariant 与 router gate 的确定性风险

- Batch-Invariant (BI): 同一条请求的 logits/输出, 不因为它和别的请求怎么组 batch、组多大而改变——逐 bit 一致。这是可复现推理 / RL rollout 对齐的硬需求。
- router gate GEMM 为什么是确定性"地雷": MoE 每层 `gate(hidden)` 出 `[M,128]` logits, top-8 选

专家。我实测（Phase B, M0-Math 48 层真实 gate）40.2% 的 token 第 8 名和第 9 名 logit 差 < 0.02，且有精确并列。意味着 logits 只要有极小扰动（不同 GPU/driver 的 split-K 累加顺序变了），就会翻转约 40% token 的专家路由 → 输出发散。所以这颗 GEMM 的"逐 bit 稳定"是 load-bearing 的。

- **persistent kernel 的性能问题**：现有 BI persistent kernel 固定 BLOCK_M=128，decode 小 M（1~12）时 grid 只有 1 个 CTA，SM 占用 0.4%，p50 约 13.8~14.9μs，是纯浪费。

§2.2 三级分派链设计

dtype	Auto 分派链	未命中/预检失败
BF16	DeepGEMM → Triton full-K → persistent	capture 前静态回退 persistent
FP16	Triton full-K → persistent	同上
FP32	per-row full-K (M≤128, FP64 预检) → persistent	预检失败/M>128 回退
不支持的 shape/布局/平台	persistent	不进候选

- 覆盖面很窄且故意窄：仅 SM90、未量化、无 bias、连续二维 [M,2048]×[2048,128] 的 router gate。Dense/QKV/O/量化 router/非 CUDA 全不碰。
- 开关：VLLM_BATCH_INVARIANT=1 + VLLM_BATCH_INVARIANT_ROUTER_GEMM=auto（默认 persistent，一个环境变量一键回滚）。
- selector 按完整 signature 静态缓存；CUDA Graph capture 前跑完依赖/JIT/输出契约/dtype 正确性/graph 预检，replay 中绝不动态换 backend（换 kernel 会破坏 graph、也破坏确定性）。

§2.3 三个后端各自的角色

① Triton full-K（我的确定性基石）

- "full-K" = 不切 K。K=2048 由单个 CTA 从 0 到 2047 固定顺序累加，无 split-K、无 atomic、无跨 CTA reduction。
- 由构造保证 batch-invariant：每个输出元素只由一个 CTA 拥有，tile 形状只决定"哪个 CTA 算哪块",从不改变 fp32 K-reduction 的顺序。我实测 BM8..BM128 各种 tile 对 persistent 逐 bit 相同 ($M \in \{1..12238\} \times \text{seed}\{0,1,2\}$, worst $|\text{maxdiff}|=0$)。
- 所以 M-based tile 分派天然 BI：小 M 用小 tile (decode 快)， $M > 2048$ fallback persistent (bit-identical，大 M 零回退)。
- fp32 版是 per-row $\text{BM}=1, \text{BN}=32, \text{BK}=64, \text{w4}, \text{s3}$ ，且必须过 FP64 $\text{rtol}=1\text{e-}5/\text{atol}=1\text{e-}6$ 预检才准入 (fp32 不宣称和 persistent 逐 bit 同，是不同算术实现)。

② DeepGEMM (H200 上更快的候选)

- 同卡微基准 decode $M=1/2/4/8/12$ ：DeepGEMM $4.74 \sim 4.86 \mu\text{s}$ vs full-K $6.83 \sim 6.95 \mu\text{s}$ vs persistent $13.8 \sim 14.9 \mu\text{s}$ 。DeepGEMM 比 full-K 快 $1.40 \sim 1.47 \times$ 。
- 小 M 自动选 BLOCK_M=16 之类，避免 persistent 固定 128 的大面积无效计算。
- 与 persistent 微基准 + 20 场景整模型逐 bit 一致。

③ persistent (生产默认 + 最终 fallback)

- 一切不满足快路径条件的情况都回到它；它 bit-identical 且久经生产。

§2.4 深挖题③：为什么是 opt-in 而不是改默认 (最能体现判断力)

结论先行：全部计划内验收都过了 (正确性、微基准、Graph、BF16/FP16 40/40 场景、TP=2 160/160、FP32 TP=1/2)，但我没把 auto 升为默认。原因是一个我自己设的独立晋级门槛没过：

- 最终同卡 GPU2 复测：DeepGEMM 相对 persistent = $2.949 \times$ ，但相对优化后 full-K 只有 $1.127 \times$ ，没到我设的 $1.2 \times$ 默认晋级线。
- 含义：full-K 已经把 decode router 打得够快了，DeepGEMM 的额外 $1.127 \times$ 不足以证明"值得引入 DeepGEMM 2.6.1 + JIT/cache 这套外部依赖 + 改默认的风险"。
- 所以交付形态 = opt-in PR，默认仍 persistent，一个环境变量灰度/回滚。只有同场终测和 acceptance 都过了，fallback 链才可能升级为默认。

这题的加分点在"我给自己设了门槛并诚实报告没过": 不是所有 5% 提升都值得改生产默认。router GEMM 只占整步 GPU busy 的一部分 (Router GPU 时间中位降 67.65%, 但整步 GPU busy 只降 5.01%, wall 中位提升 5.83%), 改默认的收益/风险比在 $1.127\times$ 这个增量上不划算。

追问预案

- **Q: Router GPU 时间降 67% 为什么端到端只有 5%?** A: Amdahl。router gate 只是每层众多算子之一, attention/MoE expert GEMM/KV 才是大头。67% 是"这颗 kernel 内部"的降幅, 不是整步。这条口径我在报告里反复强调, 不混用。
- **Q: full-K 既然 BI 又快, 为什么还留 DeepGEMM 在链里?** A: DeepGEMM 在 H200 decode 确实更快 ($1.4\times$), 且和 persistent 逐 bit 一致; 留它是给"愿意接受外部依赖换更快"的部署一个选项, 链的顺序 **DeepGEMM→full-K→persistent** 让有 DeepGEMM 环境的优先用它, 没有的退到零依赖的 full-K。
- **Q: 怎么保证 TP 两个 rank 选一样的 backend?** A: selector 按 signature 静态缓存 + 两 rank 输入 signature 相同 → 决策相同; TP=2 验收专门校验了 rank parity (160/160 + selector 决策一致)。

§2.5 深挖题④: cuBLAS 在这个 shape 上"恰好"也是 BI——那我这套还有没有意义?

这是最容易被问倒、也最能体现我理解深度的一题。

- 实测重要前提修正: 在 router 这个具体 shape **$[M, 2048] \times [2048, 128]$** bf16 上, cuBLAS (**F.linear**) 在这台 H200 (driver 570.86.15, torch2.10+cu130) 本身就是 **batch-invariant** 的——同一 needle row 在任何 M/position 下逐 bit 一致 (worst |diff|=0)。因为 $K=2048/N=128$ 太小, cuBLAS 在这些 M 下从不切 split-K。
- 所以"换 persistent→full-K"并没有修复任何在这台机器上可观测到的 nondeterminism——它是一个 **纯 $\sim 3\times$ decode 提速**, 同时保住了"由构造保证"的 BI。
- 那意义在哪? cuBLAS 的 BI 是"这台机器这个版本恰好满足", **不是它承诺的**。换 driver / 换 GPU / 换 cuBLAS 版本, 它完全可能开始切 split-K → 累加顺序变 → 翻转那 40% 近似并列的 token 路由。

full-K 是**契约级**的 BI（源码里就没有 split-K/atomic），跨硬件/驱动/版本都成立。所以正确的表述是："nd = 无约束速度下界"，不是"nd = 不确定性下界"。我把计划里"nd = nondeterministic lower bound"这个措辞主动改正了。

追问预案

- Q: 那你怎么证明 40% 近似并列这个风险是真的？ A: Phase B 在 M0-Math 48 层真实 `mlp.gate.weight` 上测的：8064 行里 40.2% 的 $|\logit_{\#8-\#9}| < 0.02$, p50 margin 0.031, min=0（精确并列）。这不是我编的阈值，是真实权重跑出来的分布。
- Q: e2e 层面 BS1 vs BSN 能逐 bit 一致吗？ A: 不能，而且这不是 router 的锅。
current(persistent)、candidate(full-K)、nd(cuBLAS) 三个变体在 $L > \sim 2048$ 都不满足 BS1==BSN，且首个 **break** 点完全一样——是 chunked-prefill / decode 注意力的 batch-composition 敏感性（attention tiling / FlashAttn decode split-KV num_splits 随 batch 变），router GEMM 既不制造也不修复它。真正能过的正控是 **current==candidate**（透明性）和 run/batch-run 自一致，这些都 PASS。这条要主动讲，否则会被"你的 BI 没做到 e2e 一致"将军。

§3 通用追问（两块都可能问）

- Q: AWQ 的 GEMV 和 GEMM 怎么分派？为什么？ A: `x.numel()/hidden < 8 && seq==1` 走 GEMV（decode, M 极小，一 warp 一 OC 足够），否则 GEMM（prefill, M 大，上 Tensor Core mma）。分界点是"M 够不够喂满一个 tile"。
- Q: split-K 和 full-K 是矛盾的吗？ A: 是两个场景。AWQ GEMM 的 split-K 是 prefill 要吞吐、可以牺牲逐 bit（用 `[split_k_iters, M, OC]` 中间 buffer 再 reduce）；Router full-K 是 decode 要确定性、故意不切 K。同一个人做两块，正好体现"什么时候该切、什么时候不能切"的判断。
- Q: double buffer 为什么能快？ $1.08\times$ 为什么不更多？ A: ping-pong 让 tile k+1 的 global→shared load 和 tile k 的 mma compute 重叠，藏 load 延迟。 $1.08\times$ 有限是因为 AWQ 权重 int4 搬得少、本就没那么 memory-bound，overlap 收益不如 fp16 GEMM 明显。
- Q: 这些 kernel 的正确性怎么保证的？ A: 两层——(1) 单测对拍 PyTorch 反量化 reference, fp16 `ATOL=1e-3`；(2) Router 侧更狠，逐 bit `torch.equal` + 1000 次 CUDA Graph

replay + 真实 gate top-8 语义 + 跨进程/跨 rank parity。决定性 gate 是 `candidate==current` 逐 bit，不是"误差够小"。

§4 红线复述（交付/合规，务必守住）

- 密钥（SwanLab / OSS AK-SK / 私有 token）只走平台环境变量注入，绝不写进任何文档/脚本/git。
- 不代替我提交/合并/关闭 PR；PR 描述写在项目内 md，由我本人手动提交。
- LLMQRT 单卡、无 TP；浅克隆无 git 时间线——这两条边界在讲的时候主动交代。
- Router GEMM 是 opt-in、默认 persistent、一键回滚——别说成"已改默认"。